

MovieHub

Web Based Movie Recommendation System

Placement Training Project

Name: V. Mokshitha Roy

Department: Electronics and Communication Engineering

College: Annamacharya Institute of Technology and Sciences, Tirupati

Application: MovieHub

1. Abstract

MovieHub is a web-based movie recommendation system built using Python and Django. It allows users to browse a multilingual movie catalogue, search and filter by genre and language, rate movies on a five-point scale, maintain a personal watchlist, and receive personalized recommendations based on their own rating history. The recommendation engine uses a content-based filtering algorithm written in plain Python — no machine learning library is required. The application also includes a Django administration panel for catalogue management, and a clean, dark-themed responsive user interface.

2. Introduction

With thousands of movies available across multiple languages and streaming platforms, discovering content that matches a user's personal taste has become genuinely challenging. Most users spend significant time browsing without finding what they want.

MovieHub addresses this by building a lightweight, self-contained recommendation website. The project was developed as a placement training exercise to demonstrate full-stack web development using the Django framework, relational database design with SQLite, HTML/CSS-based front-end development, and a basic content-based recommendation algorithm implemented from scratch in Python.

3. Problem Statement

Traditional movie platforms either require paid subscriptions or rely on opaque, ML-heavy recommendation systems. Users need a simple, browser-based system that:

- Provides a diverse, multilingual movie catalogue
- Recommends movies based on the user's own preferences
- Works without any external machine-learning library
- Requires no software installation — just a web browser
- Has an intuitive and visually appealing interface

4. Objectives

The main objectives of the project are:

- To design and implement a relational database for movies, genres, users, ratings, and watchlists using SQLite.
- To build a complete Django web application with user registration, login, and session management.
- To implement a content-based filtering algorithm in plain Python that recommends movies based on a user's rating history.
- To allow users to search, filter by genre and language, and sort the movie catalogue.

- To provide a personal watchlist feature and a “You might also like” section on every movie page.
- To configure the Django admin panel for full catalogue management.
- To design a responsive, dark-themed user interface using hand-written HTML and CSS.

5. Literature Review

Popular platforms such as Netflix, Amazon Prime Video, and IMDb use recommendation systems based on collaborative filtering (using behavior from millions of users) and hybrid models combining content signals with user history. These systems require large datasets and complex ML infrastructure.

The following concepts were studied and adapted for this project:

- Content-based filtering — recommending items similar to what a user has already liked
- Collaborative filtering — using preferences of similar users
- Genre-based tagging and language classification for movie catalogues
- Responsive web UI design patterns for media applications

MovieHub implements a simplified content-based approach that produces useful results even with a small catalogue and a single user’s history, making it well-suited for an educational project.

6. Technologies Used

Technology	Purpose	Version / Notes
Python 3	Core backend language	3.10 or later
Django	Web framework: routing, ORM, auth, templates	5.x
SQLite	Relational database (bundled with Django)	Default DB, no server needed
HTML5 + CSS3	Front-end structure and styling	Hand-written, no framework
Django Templates	Server-side HTML rendering	Built-in templating engine

7. System Architecture

The system follows the Model-View-Template (MVT) architecture provided by Django.

Frontend Layer

Responsible for all visual output including the home page (recommended and latest movies), browse page (search, filter, sort, pagination), movie detail page (rating form, similar movies), watchlist page, recommendations page, and login/signup forms. All pages are rendered server-side by Django's template engine.

Backend Layer

Handles all HTTP requests through Django view functions. The views read and write data through the ORM, call the recommendation engine for personalized results, enforce login requirements on protected pages, and return rendered templates to the browser.

Recommendation Engine

A standalone Python module (recommender.py) that builds a taste profile from genres of movies the user has rated 4 or 5 stars, scores every unrated movie by genre overlap, and returns the top matches. This requires no external library.

Database Layer

SQLite stores all persistent data: the movie catalogue (title, language, year, director, description, genres), user accounts, one rating per (user, movie) pair, and each user's watchlist.

8. Methodology

The project was developed in six steps:

Step 1 — Requirement Analysis

Features were identified and prioritized: catalogue browsing, search and filter, user authentication, star ratings, watchlist, personalized recommendations, and admin panel.

Step 2 — Database Design

An entity-relationship model was designed for Genre, Movie, Rating, and Watchlist tables with appropriate foreign keys and many-to-many relationships.

Step 3 — Backend Development

Django models, view functions, URL routes, and forms were implemented. The recommendation engine was written as a separate Python module.

Step 4 — Frontend Development

HTML templates and a hand-written CSS file were built using Django's Template Language. The dark-themed, responsive layout uses flexbox for the movie card grid.

Step 5 — Data Seeding

A custom Django management command was written to populate the database with 41 real movies across 10 languages (English, Hindi, Telugu, Tamil, Malayalam, Kannada, Korean, Japanese, Spanish, French) and 12 genres.

Step 6 — Testing

All features were tested using Django's test client and manual browser testing. A total of 12 test cases were defined and verified.

9. Features Implemented

Feature	Description
Movie Catalogue	41 movies across 10 languages and 12 genres, displayed in a responsive card grid
Search & Filter	Filter by genre and language; sort by rating, year, or title; text search by title or director
User Authentication	Registration, login, logout using Django's built-in authentication system
Star Rating System	1–5 star ratings; re-submitting updates the existing rating without creating a duplicate
Personal Watchlist	Add or remove any movie with a single click; persists in the database
Recommendations	Personalized page: genres of 4–5-star movies used to score and rank unrated candidates
You Might Also Like	Genre-overlap-based similar movies shown on every movie detail page
Django Admin Panel	Full CRUD management of movies, genres, and ratings via the built-in admin interface

10. Results and Discussion

The application was tested using Django's HTTP test client and manual browser testing. The following results were observed:

#	Test Case	Expected Result	Status
1	Home page loads for anonymous visitor	HTTP 200, newest movies shown	Pass
2	Search by title / director	Correct movies returned	Pass
3	Filter by language	Only selected-language movies shown	Pass
4	Filter by genre	Only tagged movies shown	Pass

#	Test Case	Expected Result	Status
5	User registration	Account created, user logged in	Pass
6	Submit a rating	Rating saved, average rating updates	Pass
7	Re-submit same movie rating	Existing rating updated, no duplicate	Pass
8	Add movie to watchlist	Movie appears on watchlist page	Pass
9	Remove movie from watchlist	Movie removed from watchlist page	Pass
10	Recommendations after rating Sci-Fi highly	Sci-Fi / adjacent genre movies surface first	Pass
11	Request non-existent movie ID	HTTP 404 returned	Pass
12	Admin panel access as superuser	Movies and genres fully manageable	Pass

All 12 test cases passed. The content-based filtering algorithm correctly ranked genre-matched movies higher than unrelated ones.

11. Advantages

- Self-contained: runs with only Python and Django installed, no additional server needed
- No external ML library — recommendation logic is transparent and easy to explain
- Multilingual catalogue covering 10 languages including major Indian regional languages
- Django's built-in admin saves development time for catalogue management
- Fully browser-based — no installation needed for end users
- Clean dark-themed responsive UI without any CSS framework dependency

12. Future Scope

The project can be enhanced in the following ways:

- Collaborative filtering once more users and ratings are available
- Integration with an external movie database API (e.g. TMDb) for real poster images and cast details
- Cloud deployment using PostgreSQL and Gunicorn on a production server

- User profile pages showing full rating history
- REST API (Django REST Framework) to support a companion mobile app
- AI-powered natural language search (“action movies like Inception”)
- Review and comment system for community engagement

13. Conclusion

MovieHub successfully demonstrates end-to-end web application development using Python and Django. The project delivers a working movie recommendation system with a multilingual catalogue, genre and language filtering, user authentication, star ratings, watchlist management, and a personalized recommendation engine — all without any external machine-learning library. The content-based filtering algorithm is implemented in plain Python and can be read, understood, and extended without specialist ML knowledge. All planned objectives were achieved, and the application is a strong practical demonstration of Django’s capabilities for building real-world data-driven websites.